

1 Introduction

This text is an attempt to mathematically formalize Minecraft flying machines to ultimately prove that push limit 1 flying machines do not exist. The reader is supposed to be familiar with game mechanics and jargon. The game mechanics are formal enough as they are written in code. However, we will try to create a sufficiently simplified model of the game to make the task more feasible. For example, we make these assumptions right away:

- The world is infinite in all directions, unlike actual minecraft worlds
- In this text we do not account for any precision artifacts of float numbers and work with proper real numbers ($\mathbb{R}$)
- Random number generation (RNG) is non-deterministic and truly random

2 Crafting a formal definition for flying machines

Definition 1. A *structure* is a pair of sets (B, E) , where B is a non-empty set of blocks with unique positions, and E is any set of entities. Here *blocks* and *entities* are treated as objects with position, state, and other data that would be provided by the game.

Definition 2. Let us define the *bounding box* of a structure. Denote by $\mathcal{P}$ the following set: $\{I_1 \times I_2 \times I_3 \mid I_j = (a, b) \cap \mathbb{Z}; a, b \in \mathbb{Z} \cup \{\pm\infty\}\}$. For example, $\{0, 1, 2\} \times \{0, 1\} \times \mathbb{Z}$ belongs to $\mathcal{P}$.

The bounding box of a structure S is the smallest prism from $\mathcal{P}$ by inclusion, such that it contains S .

Definition 3. A structure is *finite* if it has a bounding box with finite dimensions. Otherwise, it is *infinite*.

Definition 4. We denote by *vacuum* a completely empty world with $doDayLightCycle = false$, $doWeatherCycle = false$, $randomTickSpeed = 0$, $doMobSpawning = false$, all chunks being force loaded. The rest of the settings are default. The game difficulty may vary depending on context.

Definition 5. Let S be a structure. Given $v \in \mathbb{Z}^3$, the notation $S + v$ means a new structure obtained from S by translating all objects in it by v . If $t \in \mathbb{N}_0 := \mathbb{N} \cup \{0\}$, we denote by $\mathcal{E}_t(S)$ (or just $\mathcal{E}_t S$) a new structure obtained by evaluating S in vacuum for t game ticks.

2.1 Clarifications on random events

In the beginning of the text I postulated true randomness, but let me elaborate what that actually means. Upon creating a vacuum world, we also generate a *truly random sequence* of integers (r_n) . We then postulate that n -th call of “random” simply yields r_n . This effectively makes all game events deterministic; however, our sequence (r_n) generally lacks periodicity, unlike the pseudo random sequences that the actual game uses. This point of view is convenient for preventing certain structures from being formally periodic in time.

Proposition 1. $\{\mathcal{E}_t \mid t \in \mathbb{N}_0\}$ endowed with composition operation is a commutative monoid. That is, the composition of evaluations is associative, $\mathcal{E}_0 = Id$, and

$$\mathcal{E}_t \mathcal{E}_s = \mathcal{E}_s \mathcal{E}_t = \mathcal{E}_{t+s}.$$

2.2 Formal periodic flying machines

Definition 6. A *periodic flying machine* (or *PFM*) is any structure such that there exist $t \in \mathbb{N}, v \in \mathbb{Z}^3$, satisfying

$$\mathcal{E}_{t_0}S + v = \mathcal{E}_{t_0+t}S \quad \forall t_0 \in \mathbb{N}_0, \forall (r_n),$$

where the equality is to be understood as the equality of sets. Here (r_n) is the random sequence associated with the world.

Definition 7. A periodic flying machine is *stationary* if v can be chosen as $(0, 0, 0)$. Otherwise, it is *non-stationary* and such machines will be often referred to as *NPFM*.

Remark 1. We assume that objects in structures store enough data for the game to calculate the exact state of any structure after 1 tick using only that data and (r_n) .

Remark 2. Our definition of a PFM includes all sorts of static structures that have nothing to do with slime. Even more concerning, it excludes many actual flying machines. For example, there exists a slime machine that moves along X axis between docking stations A and B , and each time it docks with B , the station increases the X coordinate by 1. However, this does not actually make the definition worthless. The aforementioned machine is simply a disjoint union of 3 periodic flying machines: 2 docking stations and one engine. The stations are stationary periodic flying machines with $t = 1, v = (0, 0, 0)$, which happen to move upon some perturbations from outside (in vacuum they do not happen, so formally the stations are indeed stationary). The engine can be viewed as an NPFM with $t = 10, v = (1, 0, 0)$.

Remark 3. Note that according to our definition, falling sand is not a periodic flying machine. The reason is that the state of a falling block includes a lifetime variable, which strictly increases with each game tick and hence $\mathcal{E}_tS \neq S + v$ for any $t \in \mathbb{N}, v \in \mathbb{Z}^3$. Lazy chunk shenanigans are not of any use, because we assume that all chunks are force-loaded.

At this point we could introduce “formal flying machines” as disjoint unions of periodic flying machines, but this is pointless because this class would include a ridiculous amount of possible structures, because a singular block placed in vacuum acts like a stationary periodic machine, unless it is a gravity block. In practice, all slime machines that are of any interest contain at least one NPFM. This reduces the question to investigating only NPFMs.

For now let us focus on analyzing finite NPFMs.

2.3 Choosing a feasible subclass of NPFMs

The simplest example of a finite NPFM is any finite set of vertical flowing water or lava blocks. However, this is rather boring. Conventionally, let us consider only NPFMs that do not use liquids.

For the same reason, I suggest disregarding entities from discussion, at least for now. That is because they add too much complexity to an already difficult task, not to mention that there are examples such as falling minecarts, fireballs, and wither projectiles.

3 General engines

Definition 8. Define E as the set of all finite NPFMs without liquids, entities, and command blocks. The letter E was chosen because it is effectively the set of engines.

Definition 9. Given $S \in E$, define the *period of S* as the smallest number $t \in \mathbb{N}$ for which there exists $v \in \mathbb{Z}^3 \setminus \{(0, 0, 0)\}$, such that $S + v = \mathcal{E}_tS$. We will use the following notation for periods: $P(S) := t$.

3.1 TODO

It is now useful to rethink “motion”. Currently, we can only track blocks from S in intervals of $t = P(S)$ ticks. We know nothing about intermediate states of S , say, after $t - 1$ ticks, when $t > 1$. It could be air, it could be a single block of dirt. We have to figure out the way blocks move in such machines.

What if a structure S from E “moves” by disappearing and re-appearing after $P(S)$ ticks somewhere else? This isn’t far from truth: blocks that are being moved by pistons temporarily convert to b36.

Lemma 1. *Let $S \in E$. Then any non-gravity block in S changes position only after being pushed or pulled, either directly, or by other blocks being pushed or pulled.*

Proof. ...

□

Lemma 2. *Any engine must not contain immovable blocks except b36.*

Proof. ...

□

Lemma 3. *Any engine must contain at least two pistons.*

Proof. ...

□

4 Push limit 1 engines

Definition 10. Denote by E_1 the set E defined under the assumption that push limit is 1.

Lemma 4. *Any engine from E_1 must have at least one sticky piston.*

Proof. ...

□

Remark 4. Note that this lemma is wrong for larger push limits. It is easy to build an engine without sticky pistons with the default push limit of 12.